PF Final Practice Sheet (Circular Prime Level)

1. Armstrong Rotations

A number is called an Armstrong number if the sum of its digits raised to the power of the number of digits equals the number itself (e.g., 153 -> $1^3 + 5^3 + 3^3 = 153$).

Write a program that checks whether all rotations of a number are also Armstrong numbers.

2. Digit Prime Chain

Input a number. Rotate it like a circular prime, but at each rotation, remove one digit from the right. Check whether each resulting number is prime.

Example: 197 -> 197, 19, 1 -> All should be prime.

3. Self-Dividing Number

A number is self-dividing if it is divisible by each of its digits (excluding zero).

Write a function to check if a number is self-dividing.

4. Recursive Palindrome Number

Write a recursive function that checks if a number is a palindrome.

Do NOT use strings or arrays.

5. Recursive Decimal to Any Base (2 to 9)

Write a recursive function that converts a decimal number into any base from 2 to 9.

Example: decimalToBase(10, 3) -> Output: 101

6. Next Circular Prime

Write a function that finds the next circular prime greater than a given number n.

7. Magic Number

A number is called magic if repeated sum of its digits eventually leads to 1.

Example: 835 -> 8+3+5 = 16 -> 1+6 = 7 -> not magic

Example: 19 -> 1+9 = 10 -> 1+0 = 1 -> magic

8. Special Binary Sequence

Write a recursive function that checks if a binary number (entered as int) contains equal number of 0s and 1s.

9. Multiplicative Persistence

Count how many times you multiply the digits of a number until you reach a single digit.

Example: 39 -> 3×9=27 -> 2×7=14 -> 1×4=4 -> Output: 3 steps

10. Sum of All Circular Permutations

Given a number, generate all its digit-rotated versions and return the sum.

Example: 197 -> 197 + 971 + 719 = 1887

11. Pandigital Number

A number is pandigital if it contains all digits from 1 to n exactly once.

Examples: 123 -> Pandigital, 321 -> Pandigital, 1123 -> Not Pandigital